



Core idea

Euler angles describe orientation as three successive rotations—typically yaw, pitch, and roll. **Quaternions** represent the same 3D orientation using four numbers constrained to unit length. Euler angles are intuitive and easy to inspect; quaternions are better for composing, interpolating, and continuously updating rotations. ^[1] ^[2]

Euler angles and gimbal lock

Suppose an aircraft uses a yaw–pitch–roll sequence:

1. Rotate around the vertical axis: yaw.
2. Rotate around the lateral axis: pitch.
3. Rotate around the forward axis: roll.

The important detail is that these rotations are **ordered**. At a pitch of $+90^\circ$ or -90° , two of the rotation axes become aligned. Yaw and roll then describe rotation around effectively the same axis, so the representation loses one independently controllable degree of freedom. This is **gimbal lock**. ^[2] ^[3]

It is not that the physical object suddenly loses the ability to rotate. Rather, the chosen coordinate description becomes singular: different yaw/roll combinations can produce the same orientation, and small changes can cause large, unintuitive jumps in the reported angles.

Euler angles remain useful because they are:

- Human-readable: “rotate 30° yaw, 10° pitch, and 5° roll.”
- Convenient for UI controls, configuration files, and debugging.
- Appropriate when rotations are naturally limited to separate axes, such as a robot joint or a camera with bounded pitch.

Why quaternions avoid it

A unit quaternion can be written as

$$q = (x, y, z, w)$$

with the constraint

$$x^2 + y^2 + z^2 + w^2 = 1.$$

For a rotation of angle θ around unit axis $\mathbf{u} = (u_x, u_y, u_z)$:

$$q = \left(u_x \sin \frac{\theta}{2}, u_y \sin \frac{\theta}{2}, u_z \sin \frac{\theta}{2}, \cos \frac{\theta}{2} \right).$$

Unlike Euler angles, a quaternion does not apply three rotations around three sequential “gimbals.” It represents the complete orientation directly through an axis and angle embedded in a four-dimensional mathematical representation. There is therefore no special middle angle at which two coordinate axes must align.

Quaternions consequently avoid the **Euler-angle singularity** known as gimbal lock. They also support smooth interpolation, commonly through spherical linear interpolation, or **SLERP**, which is useful for animation and motion planning.^{[4] [1]}

However, quaternions are not magic:

- They still describe only three physical degrees of freedom despite using four components.
- They have a double representation: q and $-q$ encode the same orientation.
- They can accumulate numerical error, so repeatedly updated quaternions should be normalized.
- Converting them back to Euler angles can reintroduce discontinuities and apparent gimbal lock.
- A badly designed control system can still have mechanical or task-level singularities, even if its orientation representation uses quaternions.

Implications for robotics

Robotics software commonly uses quaternions internally for a robot’s orientation, pose estimation, IMU fusion, drone attitude, and end-effector motion. They are particularly valuable when orientation changes continuously over time—for example, integrating gyroscope data or interpolating a robot arm between two poses. Robotics references commonly treat quaternions as one of the standard alternatives to Euler angles, rotation matrices, and axis-angle representations.^{[5] [1]}

Practical consequences include:

- **Stable state estimation:** Sensor-fusion algorithms can update orientation without crossing an Euler-angle singularity.
- **Reliable pose interpolation:** A robot can move smoothly between orientations using SLERP or related interpolation.
- **Easy composition:** Multiplying quaternions composes rotations without manually managing a sequence of Euler-axis rotations.
- **Compact storage:** Four floating-point values are cheaper to store than a 3×3 rotation matrix, although they require normalization.
- **Better motion planning:** Interpolating quaternion orientations avoids sudden flips caused by Euler-angle wrapping.

Euler angles are still useful at the interface layer. A robot operator may want to command “yaw 20°, pitch 5°,” while the controller converts that command into a quaternion or rotation matrix

internally. A good design is therefore:

Use quaternions or rotation matrices as the authoritative internal representation; use Euler angles mainly for display, human input, and constrained joint controls.

One important distinction: a robot's **joint limits** and kinematic singularities do not disappear merely because quaternions are used. Quaternions solve the representation problem for orientation; they do not eliminate physical reachability limits or all Jacobian singularities.

Implications for game design

Game engines generally store object orientation as a quaternion or equivalent internal representation, while exposing Euler angles in the editor because they are easier for designers to understand. Unity, for example, documents quaternion rotations as avoiding gimbal lock and supporting compact, interpolatable rotation data.^[6]

Quaternions are especially useful for:

- Camera orientation.
- Aircraft and spacecraft controls.
- Character and weapon animation.
- Physics-based rigid bodies.
- Networked objects whose rotations must be interpolated.
- Turrets, vehicles, and objects that can rotate freely in all directions.
- Blending between animation poses.

A common game-development mistake is to keep reading an object's current Euler angles, modifying them every frame, and writing them back. This can create angle wrapping, sudden flips, and apparent gimbal lock. Instead, maintain a quaternion as the source of truth and apply incremental rotations to it:

```
orientation = delta_rotation * orientation;  
orientation.normalize();
```

For a first-person camera, you may still intentionally use yaw and pitch as gameplay variables:

```
yaw    = yaw + mouse_x;  
pitch = clamp(pitch + mouse_y, -89°, +89°);  
orientation = quaternion(yaw) * quaternion(pitch);
```

That is not a contradiction. The game is deliberately restricting pitch before constructing the quaternion, so the user-facing control model remains intuitive while the resulting orientation is handled robustly.

Practical rule of thumb

Task	Prefer
Designer-facing rotation fields	Euler angles
Debugging and readable configuration	Euler angles
Continuous orientation updates	Quaternions
Smooth rotation interpolation	Quaternions / SLERP
Transforming many vectors or points	Rotation matrices or engine-optimized forms
Individual hinged robot joints	A single joint angle
Free 3D rigid-body orientation	Quaternions

In short: Euler angles are a convenient description of three ordered rotations, but their coordinate system can become singular. Quaternions represent orientation without that singularity, making them the safer internal choice for robotics and free-form game motion—while Euler angles remain valuable for human interaction and constrained controls.



1. https://ethz.ch/content/dam/ethz/special-interest/mavt/robotics-n-intelligent-systems/asl-dam/documents/lectures/robot_dynamics/RD2_Quaternions.pdf
2. <https://www.chrobotics.com/files/docs/an-1006-understandingquaternions.pdf>
3. <https://siliconwit.com/education/robotics/orientation-quaternions/>
4. <https://saeed1262.github.io/blog/2024/quaternion/>
5. https://rocco.faculty.polimi.it/IAR/Robot_kinematics.pdf
6. <https://discussions.unity.com/t/is-quaternion-really-do-not-suffer-from-gimbal-lock-714883/714883>
7. <https://www.mysimulator.uk/content/articles/rotation-matrices-quaternions-robotics.html>
8. <https://gamedev.stackexchange.com/questions/30162/calculating-up-vector-to-avoid-gimbal-lock-using-euler-angles>
9. <https://www.chrobotics.com/library>
10. <https://bugnet.io/blog/how-to-fix-unity-gimbal-lock-from-euler-angle-rotation>
11. https://pulli.me/projects/gimbal_lock/
12. <https://notes.suhaib.in/docs/tech/latest/why-quaternions-revolutionized-3d-rotation-and-what-euler-angles-got-wrong/>
13. <https://www.dm.unibo.it/~ferri/hm/tesi/tesiSenzaniPezzi.pdf>
14. <https://faculty.cc.gatech.edu/~hic/4632B-07/kinematics-draft.pdf>
15. <https://stackoverflow.com/questions/28776788/quaternion-reaching-gimbal-lock>